AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 7: Triangular Systems; Backward Stability of Back Substitution

Prof. Xiangmin Jiao

Stony Brook University

Outline

1. **Triangular Systems (MC §3.1)**
2. Backward Stability of Back Substitution (NLA §17)

Triangular Systems

- A matrix $G = (g_{ij})$ is *lower triangular* if $g_{ij} = 0$ whenever $i < j$.

$$G = \begin{bmatrix} g_{11} & 0 & 0 & \cdots & 0 \\ g_{21} & g_{22} & 0 & \cdots & 0 \\ g_{31} & g_{32} & g_{33} & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & 0 \\ g_{n1} & g_{n2} & g_{n3} & \cdots & g_{nn} \end{bmatrix}.$$

  - It is *strictly lower triangular* if $g_{ij} = 0$ whenever $i \leq j$.
  - It is *unit lower triangular* if $g_{ij} = 0$ whenever $i < j$ and $g_{ii} = 1$.
- Similarly, G is *upper triangular* if $g_{ij} = 0$ whenever $i > j$.
- A *triangular matrix* is a matrix that is either upper or lower triangular.
- A triangular matrix $G \in \mathbb{R}^{n \times n}$ is nonsingular if and only if $g_{ii} \neq 0$ for $i = 1, \ldots, n$.

Lower-Triangular Systems

- Consider the system

$$Gy = b,$$

  where G is a nonsingular, lower-triangular matrix.

- We can solve the system by

$$\begin{aligned}
y_1 &= b_1/g_{11} \\
y_2 &= (b_2 - g_{21}y_1)/g_{22} \\
&\vdots \\
y_i &= (b_i - g_{i1}y_1 - g_{i2}y_2 - \cdots - g_{i,i-1}y_{i-1})/g_{ii} \\
&= \left(b_i - \sum_{j=1}^{i-1} g_{ij}y_j\right)/g_{ii}.
\end{aligned}$$

Forward Substitution

Forward substitution (y overwrites b)

for $i = 1 : n$
  for $j = 1 : i - 1$
    $b_i \leftarrow b_i - g_{ij} b_j$
  $b_i \leftarrow b_i / g_{ii}$

- This algorithm is *row oriented*, as it accesses G by rows.
- It may cause an error (or raise an exception) if $g_{ii} = 0$.
- The number of operations is $\sum_{i=1}^{n} \sum_{j=1}^{i-1} 2 = 2 \sum_{i=1}^{n} (i-1) = n(n-1) \approx n^2$.

Column-Oriented Forward Substitution

- We can reorder the loops and obtain a column-oriented algorithm.

Forward substitution (y overwrites b)

for $j = 1 : n$
  $b_j \leftarrow b_j / g_{jj}$
  for $i = j + 1 : n$
    $b_i \leftarrow b_i - g_{ij} b_j$

- The number of operations is again $\approx n^2$.
- In practice, the column-oriented algorithm is faster if G is stored in a column-oriented fashion.
- Like matrix-matrix multiplication, performance can be improved using block-matrix operators.

Exploiting Leading Zeros

- If $b_1 = b_2 = \cdots = b_{k-1} = 0$, how can we change the algorithm to reduce operations?
- In the row-oriented algorithm: let $i = k, \ldots, n$ and $j = k, \ldots, i-1$.
- In the column-oriented algorithm: let $j = k, \ldots, n$.
- This saves floating-point operations, especially if k is large.
  - Example: computing the inverse of a lower-triangular matrix.

Upper-Triangular Systems

- Consider the system
$$Uy = b,$$
where $U \in \mathbb{R}^{n\times n}$ is nonsingular and upper triangular.
- We solve the system from bottom to top:
$$\begin{aligned} y_n &= b_n/u_{nn} \\ y_{n-1} &= (b_{n-1} - u_{n-1,n}y_n)/u_{n-1,n-1} \\ &\vdots \\ y_i &= (b_i - u_{in}y_n - u_{i,n-1}y_{n-1} - \cdots - u_{i,i+1}y_{i+1})/u_{ii} \\ &= \left(b_i - \sum_{j=i+1}^{n} u_{ij}y_j\right)/u_{ii}. \end{aligned}$$
- This is *back substitution*, and it has the same cost as forward substitution.

Outline

1. Triangular Systems (MC §3.1)
2. **Backward Stability of Back Substitution (NLA §17)**

Backward Stability of Back Substitution

- Solve $Rx = b$ using back substitution:

$$\begin{bmatrix} r_{11} & \cdots & r_{1n} \\ & \ddots & \vdots \\ & & r_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}.$$

- Algorithm: for $j = n$ down to 1:

$$x_j = \left(b_j - \sum_{k=j+1}^{n} x_k r_{jk} \right) / r_{jj}.$$

- Back substitution is backward stable:

$$(R + \delta R)\tilde{x} = b, \quad \|\delta R\| / \|R\| = \mathcal{O}\left(\epsilon_{\text{machine}}\right).$$

  Furthermore, each component of δR satisfies

$$\frac{|\delta r_{ij}|}{|r_{ij}|} \leq n\epsilon_{\text{machine}} + \mathcal{O}\left(\epsilon_{\text{machine}}^2\right).$$

- We demonstrate the proof for $n = 1, 2, 3$ and then generalize to n.

Proof of Backward Stability ($n = 1$)

- For $n = 1$, the algorithm is simply one floating-point division. Using the floating-point axiom, we get
$$\tilde{x}_1 = b_1 \oslash r_{11} = \frac{b_1}{r_{11}}(1 + \epsilon_1) = \frac{b_1}{r_{11}(1 + \epsilon_1')},$$
where $|\epsilon_1| \leq \epsilon_{\text{machine}}$ and $|\epsilon_1'| \leq \epsilon_{\text{machine}} + \mathcal{O}\left(\epsilon_{\text{machine}}^2\right)$.
- Therefore, we have computed the exact solution to a perturbed problem:
$$(r_{11} + \delta r_{11})\tilde{x}_1 = b_1 \quad \text{with} \quad \frac{|\delta r_{11}|}{|r_{11}|} \leq \epsilon_{\text{machine}} + \mathcal{O}\left(\epsilon_{\text{machine}}^2\right).$$

Proof of Backward Stability ($n = 2$)

- For $n = 2$, we first solve for $\tilde{x}_2$ as before. Then, we compute $\tilde{x}_1$:

$$\begin{aligned}
\tilde{x}_1 &= (b_1 \ominus (\tilde{x}_2 \otimes r_{12})) \oslash r_{11} \\
&= \frac{(b_1 - \tilde{x}_2 r_{12}(1+\epsilon_2))(1+\epsilon_3)}{r_{11}}(1+\epsilon_4) \\
&= \frac{b_1 - \tilde{x}_2 r_{12}(1+\epsilon_2)}{r_{11}(1+\epsilon_3')(1+\epsilon_4')} \\
&= \frac{b_1 - \tilde{x}_2 r_{12}(1+\epsilon_2)}{r_{11}(1+2\epsilon_5)},
\end{aligned}$$

  where $|\epsilon_2|, |\epsilon_3|, |\epsilon_4| \le \epsilon_{\text{machine}}$ and $|\epsilon_3'|, |\epsilon_4'|, |\epsilon_5| \le \epsilon_{\text{machine}} + \mathcal{O}\left(\epsilon_{\text{machine}}^2\right)$.

- Again, this is an exact solution to $(R + \delta R)\tilde{x} = b$ with

$$\begin{bmatrix} \frac{|\delta r_{11}|}{|r_{11}|} & \frac{|\delta r_{12}|}{|r_{12}|} \\ & \frac{|\delta r_{22}|}{|r_{22}|} \end{bmatrix} = \begin{bmatrix} 2|\epsilon_5| & |\epsilon_2| \\ & |\epsilon_1| \end{bmatrix} \le \begin{bmatrix} 2 & 1 \\ & 1 \end{bmatrix} \epsilon_{\text{machine}} + \mathcal{O}\left(\epsilon_{\text{machine}}^2\right).$$

Proof of Backward Stability ($n = 3$)

- For $n = 3$, we solve for $\tilde{x}_3$ and $\tilde{x}_2$ as before. Then, we compute $\tilde{x}_1$:

$$
\begin{aligned}
\tilde{x}_1 &= [b_1 \ominus (\tilde{x}_2 \otimes r_{12}) \ominus (\tilde{x}_3 \otimes r_{13})] \oslash r_{11} \\
&= \frac{[(b_1 - \tilde{x}_2 r_{12}(1+\epsilon_4))(1+\epsilon_6) - \tilde{x}_3 r_{13}(1+\epsilon_5)](1+\epsilon_7)}{r_{11}(1+\epsilon_8')} \\
&= \frac{b_1 - \tilde{x}_2 r_{12}(1+\epsilon_4) - \tilde{x}_3 r_{13}(1+\epsilon_5)(1+\epsilon_6')}{r_{11}(1+\epsilon_6')(1+\epsilon_7')(1+\epsilon_8')}.
\end{aligned}
$$

That is, $(R + \delta R)\tilde{x} = b$ with

$$
\begin{bmatrix}
\frac{|\delta r_{11}|}{|r_{11}|} & \frac{|\delta r_{12}|}{|r_{12}|} & \frac{|\delta r_{13}|}{|r_{13}|} \\
 & \frac{|\delta r_{22}|}{|r_{22}|} & \frac{|\delta r_{23}|}{|r_{23}|} \\
 & & \frac{|\delta r_{33}|}{|r_{33}|}
\end{bmatrix}
\leq
\begin{bmatrix}
3 & 1 & 2 \\
 & 2 & 1 \\
 & & 1
\end{bmatrix}
\epsilon_{\text{machine}} + \mathcal{O}\left(\epsilon_{\text{machine}}^2\right).
$$

Proof of Backward Stability (General n)

- A similar analysis for general n yields the pattern

$$\frac{|\delta R|}{|R|} \leq W\epsilon_{\text{machine}} + \mathcal{O}\left(\epsilon_{\text{machine}}^2\right),$$

where W is (shown for $n = 5$)

$$\underbrace{\begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ & 0 & 1 & 1 & 1 \\ & & 0 & 1 & 1 \\ & & & 0 & 1 \\ & & & & 0 \end{bmatrix}}_{\otimes} + \underbrace{\begin{bmatrix} 1 & & & & \\ & 1 & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix}}_{\oslash} + \underbrace{\begin{bmatrix} 4 & 0 & 1 & 2 & 3 \\ & 3 & 0 & 1 & 2 \\ & & 2 & 0 & 1 \\ & & & 1 & 0 \\ & & & & 0 \end{bmatrix}}_{\ominus},$$

which results in

$$\begin{bmatrix} 5 & 1 & 2 & 3 & 4 \\ & 4 & 1 & 2 & 3 \\ & & 3 & 1 & 2 \\ & & & 2 & 1 \\ & & & & 1 \end{bmatrix}.$$